Google Maps

Rev. 2.2 del 01/03/2023

Introduzione alla geolocation …… 2

Mappe statiche 2D …… 2
Mappe statiche 3D (street view) …… 3
Collegamento dinamico alle google maps …… 3

Le API v3 per la gestione delle Google Maps …… 4
- Le classi Map e LatLng …… 4
- La classe Marker …… 5
- La classe InfoWindow …… 6
- Personalizzazione dei pulsanti di controllo …… 8
- Come nascondere le etichette delle google maps …… 9
- La classe Geocoder …… 10
- La visualizzazione di una route …… 12
- Distanza fra due punti …… 13
- Tempo di percorrenza …… 13

Lettura della posizione corrente …… 14

Geolocation

Il sistema **WGS84** (World Geodetic System del 1984) è un sistema di posizionamento e navigazione satellitare civile che, attraverso una rete dedicata di satelliti artificiali in orbita, fornisce ad un **ricevitore GPS** (**Global Positioning System**) informazioni sulle **sue** coordinate geografiche ed orario, in ogni condizione meteorologica, ovunque sulla Terra ove vi sia un contatto privo di ostacoli.

Il sistema GPS è gestito dal governo degli Stati Uniti d'America ed è liberamente accessibile da chiunque sia dotato di un ricevitore GPS. Il sensore GPS riceve segnali radio dai satelliti mediante un'antenna GPS incorporata e deduce la distanza dal satellite sulla base del tempo di percorrenza del segnale.
Il sensore può calcolare la sua posizione quando è stabilita la distanza da almeno quattro satelliti.
Più satelliti vengono rilevati, maggiore è l'accuratezza.

Nel caso di un PC privo di sensore, la posizione può essere dedotta in modo approssimato dall'indirizzo IP della connessione internet.

Le coordinate GPS sono espresse mediante una coppia di numeri decimali, entrambi con una precisione di 15 cifre decimali (double) di cui occorre usarne sempre **almeno 7.** Con 7 cifre decimali (bassa risoluzione) si ha una risoluzione di **1 metro** che è comunque decisamente inferiore rispetto alla risoluzione con cui il sensore GPS riesce a calcolare la distanza dai satelliti ed esprimere le coordinate stesse. Questa risoluzione è di circa 7/8 metri. Le coordinate espresse dal ricevitore sono:

- **latitudine** (**0°=equatore, num positivi = sopra l'equatore**, **num negativi = sotto l'equatore**)
  (polo nord = 90°, polo sud = -90°)
- **longitudine** rispetto a Greenwich (**numeri positivi = est**, **numeri negativi = ovest**)

Utilizzo delle Google Static Maps

Le mappe statiche sono semplicemente delle "fotografie" che possono essere richieste ad un preciso link pubblico di Google e visualizzate nella pagina come una qualsiasi altra immagine, ad esempio all'interno dell'attributo **`src`** di un tag **`<img>`**
Possono essere utilizzate ad esempio per indicare la posizione della propria sede aziendale.

Il link base per accedere alle mappe statiche è il seguente:

`http://maps.googleapis.com/maps/api`

a cui occorre concatenare il tipo di risorsa richiesta

`/staticmap`	Mappe Statiche 2D
`/streetview`	Immagini StreetView 3D

In coda alla risorsa occorre concatenare una serie di parametri UrlEncoded che differiscono leggermente nei due casi.

Parametri Comuni

Tutti scritti rigorosamente in **minuscolo**

key API key di identificazione.

maptype Indica il tipo di mappa da visualizzare fra le seguenti: (scritte in **minuscolo**)

`roadmap`	mappa stradale (default)
`hybrid`	mappa satellitare ibrida comprensiva di etichette e informazioni stradali
`satellite`	mappa satellitare pura priva di etichette e informazioni stradali
`terrain`	mappa fisica del territorio

size Dimensioni dell'area da visualizzare (base x altezza, es 800 x 600). Deve coincidere con le dimensioni del contenitore in cui l'immagine viene visualizzata.

zoom Definisce il valore di zoom tra 1 e 25 (i valori maggiori non provocano variazioni rispetto a 25)
zoom = 1 mostra l'intero globo. Valori normalmente utilizzati : 15 / 17
Può anche assumere valori **decimali** contenenti il puntino.

format Formato dell'immagine tra JPG (default) GIF e PNG

center definisce dove deve essere centrata la mappa.
Può essere espressa come coordinate GPS oppure tramite il nome della località.

Vallauri : latitudine Nord `44.5557763`, longitudine Est `7.7347183`
Comune di Cavallermaggiore : `44.7077582, 7.6877771`
Colosseo : `41.890617421405366, 12.493300437927246`

Parametri Relativi allo StreetView

center è sostituito da **location**. **zoom** non è riconosciuto.
Sono invece disponibili i seguenti tre parametri aggiuntivi:

heading Angolazione dell'obiettivo sull'asse orizzontale verso destra o versa sinistra fra 0 e 360°
90° indica est, 180° indica sud, 270 indica ovest (rotazione antioraria).

pitch angolazione dell'obiettivo verso l'alto (numeri positivi) oppure verso il basso (numeri negativi).
espressa in gradi. 0°=orizzontale (max 30°)

fov **zoom** del'obiettivo (distanza focale) tra 20 e 200. Valori piccoli indicano che siamo **vicini** al soggetto da fotografare, e quindi l'immagine risulta maggiormente ingrandita. Typico: **100**.

Se si utilizza il nome alfanumerico invece delle coordinate LAT/LNG occorre modificare il valore heading

markers

Consente di definire dei marcatori da posizionare sulla mappa per indicare qualcosa.
Per ogni marcatore occorre specificare le coordinate o l'indirizzo alfanumerico della località.

Per i vari marcatori possono essere impostati alcuni stili separati dal pipe **|** ed univoci per tutti i marcatori. Gli attributi utilizzabili sono i seguenti:

color = colore del marker(accettato soltanto il nome alfanumerico)
size = dimensione fra big, mid, normal, small, tiny,
label = carattere alfanumerico da visualizzare all'interno del marker. E' accettato un **singolo carattere scritto in maiuscolo**. Diversamente viene inserito automaticamente un semplice puntino.

Per ultimo devono essere indicate le coordinate o l'indirizzo alfanumerico che possono essere replicati più volte in modo da creare più marcatori (tutti uguali). Es:

```
"markers" :
"color:blue|size:big|label:V|via san michele 68, fossano|via san michele 76, fossano"
```

Collegamento dinamico alle google maps

All'interno di un **`iframe`** si può inserire un collegamento diretto alle google maps con il seguente **`src`** :

```
src="https://www.google.it/maps/embed?nomeParametro=valore"
```

con una lunga lista di parametri che possono essere copiati direttamente dalle google maps :
Menù / Condividi e incorpora mappa / incorpora mappa.

Dal 2014 i vari parametri sono stati accorpati in un unico parametro **pb** di tipo stringa e suddivisi da un !

Le API v3 per la gestione delle Google Maps

Data di rilascio ufficiale V 1 : **13 novembre 2013**
V **3.32** **13 febbraio 2018**

Sono un insieme di API che consentono la gestione delle google maps '**da codice'** con diverse possibilità di personalizzazione. Hanno un peso molto ridotto (35k) sono compatibili con tutti i dispositivi mobile. Per poter utilizzare all'interno di una pagina HTML occorre inserire il seguente link:

```
<script src="https://maps.googleapis.com/maps/api/js?key=***"> </script>
```

Gli oggetti Point, LatLng, MapOptions e Map

Tutte le classi sono contenute all'interno del namespace **google.maps**

Point

Si tratta di un semplice json contenente i due campi **lat** e **lng**.
Spesso accettato come parametro alternativo a LatLng

```
let point = {"lat":44.7077582, "lng":7.6877771}
```

google.maps.LatLng

Oggetto vero e proprio. All'interno di LatLng **lat** e **lng** sono **campi privati** (non visibili esternamente).
L'oggetto espone i seguenti metodi:

- un **costruttore** che accetta come parametri due numeri float indicanti le coordinate GPS ma che NON accetta come parametri indirizzi testuali. Esempio:
  `let posizione = new google.maps.LatLng(44.7077582, 7.6877771);`
- **.lat()** metodo che restituisce la latitudine del punto corrente
- **.lng()** metodo che restituisce la longitudine del punto corrente
- **.toJSON()** Restituisce un JSON Point (che potrà eventualmente essere serializzato)
- **.toString()** restituisce una stringa con solo le due coordinate: (44.5557763, 7.7347183)

google.maps.MapOptions

E' un semplice json che consente di configurare alcune impostazioni di visualizzazione della mappa.
I campi più importanti da impostare sono **due** (per mapTypeId va bene il default ROADMAP)

center centro mappa. Accetta come parametro sia un oggetto **LatLng** sia un semplice json **Point** ma non accetta un indirizzo testuale che deve pertanto essere convertito usando l'oggetto Geocoder.

zoom E' lo stesso delle Mappe Statiche, tra 1 (globo) e 25 (max zoom). Può anche assumere valori decimali contenenti il puntino. Deve essere impostato esplicitamente (non esiste un valore di default)

mapTypeId Accetta un oggetto di tipo `google.maps.MapTypeId` i cui possibili valori sono gli stessi delle mappe statiche scritti però interamente in maiuscolo:
google.maps.MapTypeId.ROADMAP (default) / TERRAIN oppure HYBRID / SATELLITE (senza etichette)

google.maps.Map

E' l'oggetto principale che consente di disegnare fisicamente la mappa.
Si aspetta due parametri:

- **Un puntatore javascript** ad un tag **DIV** esistente dove disegnare la mappa (il quale dovrà avere una altezza ben definite altrimenti assume altezza 0 e la mappa non viene disegnata
- Un **json mapOptions**

```
window.onload = function(){
      let div = document.getElementById("wrapper");
      let posizione = new google.maps.LatLng(44.7077582,7.6877771);
      let mapOptions = {
          center:  point/LatLng,
          zoom: 16,
          mapTypeId: google.maps.MapTypeId.HYBRID   // default=ROADMAP
      };
      let googleMap = new google.maps.Map(div, mapOptions);
}
```

I valori delle option possono essere letti/modificati anche in un 2° tempo tramite i seguenti metodi:

```
googleMap.setCenter(currentPoint)
googleMap.setZoom(13)
googleMap.setMapTypeId(google.maps.MapTypeId.TERRAIN)
let pos = googleMap.getCenter()
let zoom = googleMap.getZoom()
googleMap.setCenter(marcatore.getPosition())
```

`googleMap.panTo(currentPoint)` è simile a **`setCenter()`** ma esegue lo spostamento verso la nuova posizione tramite una piccola animazione (smoothly, liscia)

L'oggetto `Marker`

Consente di visualizzare uno o più Marker sulla Mappa. Ha un solo parametro `markerOptions`.

```
let markerOptions = {
      map:googleMap,
      position:posizione,
      title:"IIS Vallauri",    // tooltip
      icon: 'img/hotfoodcheckpoint.png',   // simple icon
      label:'hotfood',   // poco leggibile
      animation: google.maps.Animation.DROP,  // .BOUNCE oscilla
      draggable:true,   // rende il marcatore trascinabile
      zIndex: 3,        // in caso di marcatori vicini/sovrapposti
}

let marcatore1 = new google.maps.Marker(markerOptions)
```

- Il campo **`position`** accetta come parametro sia un oggetto **`LatLng`** sia un semplice json **`Point`** ma non accetta un indirizzo testuale che deve pertanto essere convertito usando l'oggetto Geocoder
- Il campo **`icon`** consente di specificare una icona personalizzata.
- Il campo **`label`** viene visualizzato al centro dell'icona e accetta non solo una singola lettera maiuscola (come nelle prime versioni), ma una generica stringa che viene visualizzata sovrapposta all'icona in modo **centrato** (posizione che può comunque essere leggermente modificata**).** Sulla **`label`** è possibile assegnare tutte le property css relative al testo (color, fontWeight, fontSize, etc) :

  ```
  label: {fontSize:'9pt',  text:"myHome"}
  ```

- Se non si specificano nè **`icon`** nè **`label`**, viene utilizzata una **`icona di default`** a forma di palloncino con colore di sfondo rosso e pallino nero centrale (che non sembra modificabile)

Invece di definire le varie proprietà in modo statico al momento dell'istanza del Marker, è anche possibile leggerle / modificarle in un secondo tempo tramite i metodi corrispondenti:

```
marcatore1.setPosition(newPosition)
marcatore1.getPosition()
marcatore1.setMap(null);     // per eliminare il marcatore dalla mappa
```

Impostazione delle complex icon

E' possibile personalizzare ulteriormente *dimensioni*, *posizione* e *offset* delle icone utilizzate come marcatori. Le chiavi da utilizzare sono le seguenti:

```
let icon1 = {
    url: 'icone/hotfoodcheckpoint.png',
    scaledSize: new google.maps.Size(16, 19),  // dimensioni
    origin: new google.maps.Point(0,0),        // file con icone multiple
    labelOrigin:{x:12, y:40},
    anchor: new google.maps.Point(30, 40),
}
icon:icon1
```

scaledSize : rappresenta la dimensione con cui visualizzare l'icona. Affinchè l'icona non venga deformata occorre verificare le dimensioni dell'icona (ad esempio con Paint) ed applicare delle dimensioni proporzionali. Se si omette, l'icona viene visualizzata con le sue dimensioni reali.

origin : Consente di traslare la lettura dell'icona all'interno di un file quando si hanno più icone memorizzate tutte nello stesso file. Il default ovviamente è 0,0

labelOrigin : rappresenta un offset di posizionamento della label rispetto all'icona stessa

anchor : rappresenta un offset rispetto alla posizione definita da position. Per spostare leggermente il marcatore. Nell'esempio 30px verso sinistra e 40px verso l'alto. Attenzione però che il valore è espresso in **px**, quindi in caso di zoom falsa completamente la posizione. Va bene se la mappa non è zoommabile

Dove trovare le icone

mapicons.mapsmarker.com Presenta una infinità di icone suddivise in gruppi e scaricabili per gruppi. Per ogni icona sono presenti 8 diverse versioni. E' possibile personalizzabile il colore di sfondo

flaticon.com e **icons8.com** consentono di impostare le dimensioni dell'icona

Marker Click

L'oggetto Marker dispone di un metodo **addListener()** all'interno del quale è disponibile il puntatore **this** che punta al marker medesimo

```
marker1.addListener('click', function() {
        console.log(this.title, this.position)
});
```

L'oggetto `InfoWindow`

In corrispondenza del click su un marcatore, si può aprire una `InfoWindow` contenente informazioni aggiuntive sul punto indicato dal marcatore. Ha un solo parametro `infoWindowOptions` all'interno del quale occorre definire un tag DIV contenente l'intero HTML che si intende visualizzare all'interno della infoWindow.

Il contenuto HTML può essere passato

- sia come **stringa** HTML diretta
- sia come puntatore java script ad un **HTML Element** a partire da una costruzione dinamica

```
let html =
    `<div id="infoWindow">
          <h2> ITIS Vallauri </h2>
          <img src="img/vallauri.jpg">
          <p>coordinate GPS: ${position.toString()} </p>
    </div>`
let infoWindowOptions = {
    content : html,
    width : 150
}
let infoWindow = new google.maps.InfoWindow(infoWindowOptions);

// oppure
let div = $("<div>").prop("id", "infoWindow")
    $("<h2>").text("ITIS Vallauri").appendTo(div)
    $("<img>").prop("src", "img/vallauri.jpg">).appendTo(div)
    $("<p>").text(`coordinate GPS: ${position.toString()}`).appendTo(div)
let infoWindowOptions = {
    content : div.get(0),
    width : 150
}
let infoWindow = new google.maps.InfoWindow(infoWindowOptions);
```

Apertura della InfoWindow

La **infoWindow** può essere aperta in corrispondenza del click sul marcatore in due modi :

1) facendo uso del metodo **addListener()** dell'oggetto Marker :

```
marker1.addListener('click', function() {
        infoWindow.open(googleMap, marker1);
});
```

Il secondo parametro di open indica il marcatore presso il quale aprire la infoWindow (che di solito, ma NON necessariamente, coincide con il marker sul quale si è fatto click).

2) Utilizzando il metodo **google.maps.event.addListener**(marker1,'click',function (){

```
        infoWindow.open(googleMap, marker1);
})
```

NOTA: **Eventuali procedure** richiamate da pulsanti interni alla infoWindow **devono** essere posizionate fuori dal documentReady() altrimenti non saranno visibili dall'html.

Intercettazione della Chiusura della InfoWindow

Per intercettare la chiusura della infoWindow tramite la X in alto a destra si può utilizzare il seguente evento da scriversi **DOPO** aver istanziato la **infoWindow**

```
google.maps.event.addListener(infoWindow,'closeclick',function(){})
```

Chiusura automatica dell'ultima InfoWindow aperta

In corrispondenza dell'apertura di una infoWindow si può provvedere a chiudere la infoWindow precedente nel seguente modo, dichiarando una variabile globale **`let lastWindow=null;`**

```
marker1.addListener("click", function () {
    if (lastWindow) lastWindow.close();
    infoWindow.open(googleMap, this);
    lastWindow=infoWindow;
})
```

Come nascondere le etichette delle google maps

Le numerosissime etichette (labels) presenti sulle google maps possono talvolta interferire con le nostre. Cioè se ad esempio un franchising vuole pubblicare sul proprio sito una mappa con tutti i propri negozi, questi risulterebbero molto più visibili eliminando tutte le altre etichette preesistenti.

Per ottenere questo risultato è sufficiente aggiungere le seguenti opzioni all'interno delle **`mapOptions`**:

```
styles: [
  {
   "featureType": "poi",     /* point of interest */
   "stylers": [
     { "visibility": "off" }
   ]
  }
]
```

Possono anche essere nascosti soltanto alcuni specifici point of interest:

```
styles: [
  {
   "featureType": "poi.park",
   "stylers": [ { "visibility": "off" }]
  },
  {
   "featureType": "poi.medical",
   "stylers": [ { "visibility": "off" }]
  },
]
```

- Se non si specifica `featureType` il default è ***all***, cioè nasconda tutto
- Oltre a `poi` sono disponibili anche `road, landscape, administrative`
- Dentro `stylers`, oltre alla `visibility`, si possono impostare anche `color e saturation`

Il seguente esempio nasconde indistintamente TUTTE le labels compresi ad es anche i nomi delle strade

```
styles: [
  {
   "elementType": "labels",
   "stylers": [ { "visibility": "off" }]
  },
]
```

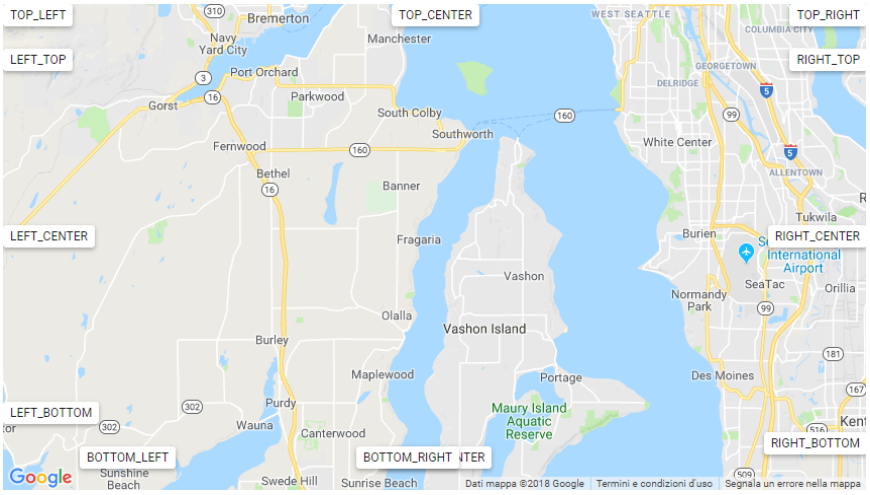
Il seguente link riporta l'elenco completo di tutte le opzioni disponibili
`https://developers.google.com/maps/documentation/javascript/style-reference?hl=it`

Personalizzazione dei pulsanti di controllo

All'interno delle opzioni precedenti è possibile aggiungere alcune opzioni che consentono il posizionamento personalizzato dei pulsanti di controllo della mappa.

```
// disabilito tutti i pulsanti di controllo
disableDefaultUI:true,

// Pulsanti switch Mappa/Satellite (il chek Rilievo visualizza il TERRAIN)
mapTypeControl: true,
mapTypeControlOptions: {
   style: google.maps.MapTypeControlStyle.HORIZONTAL_BAR, // default
   style: google.maps.MapTypeControlStyle.DROPDOWN_MENU,  // verticale
   position: google.maps.ControlPosition.TOP_LEFT,
},

// Omino StreetView
streetViewControl: true,
streetViewControlOptions: {
   position: google.maps.ControlPosition.RIGHT_CENTER
},

// Pulsanti di zoom + -
zoomControl: true,
zoomControlOptions: {
   // style: deprecata,
   position: google.maps.ControlPosition.RIGHT_CENTER
},

// Pulsante FullScreen
fullscreenControl: true,
fullscreenOptions: {
   // Non ha opzioni, utilizza una posizione fissa in alto a destra
},
// Visualizza nella riga di stato in basso a destra un fattore di scala
// per default è disabilitato
scaleControl: true,
scaleControlOptions: {
   // Non ha opzioni }
```

Le varie costanti di posizionamento sono dettagliate nella seguente mappa:



L'oggetto `Geocoder`

L'oggeto Geocoder resituisce le coordinate GPS di un indirizzo alfanumerico (es "Fossano, Via San Michele"). In realtà restituisce non solo le coordinate GPS ma tutte le informazioni di localizzazione dell'indirizzo indicato.

L'oggetto è pensato per indirizzi esistenti ed univoci. Per indirizzi incompleti occorre utilizzare l'oggetto **Place** che verifica l'esistenza di un indirizzo ed i possibili match.

```
let geocoder = new google.maps.Geocoder()
geocoder.geocode({geocodingOptions}, function(results, status){})
```

Il metodo `geocoder.geocode()` si aspetta due parametri:

1) un json geocodeOptions =:

```
{   address: string,
    location: LatLng,
    placeId: string,
    bounds: LatLngBounds,
    componentRestrictions: GeocoderComponentRestrictions,
    region: string
}
```

dove i primi tre campi sono alternativi fra loro. Cioè è possibile passare al metodo `geocode`:

- una stringa alfanumerica relativa alla località da ricercare,
- il corrispondente oggetto **LatLng**,
- il corrispondente **placeId** (ogni luogo delle google maps è identificato da un placeId univoco).

Gli ultimi campi sono facoltativi e consentono di restringere il campo di ricerca.
Nella maggior parte dei casi il geocodeOptions si riduce al solo campo ***address***

2) Il secondo parametro è una funzione di ***callback*** richiamata al termine del geocoding.
A questa funzione di callback vengono iniettati due parametri:

- **`status`** = google.maps.GeocoderStatus.OK / NOK stato della risposta
- **`results`** che è un **vettore enumerativo di oggetti GeocoderResult** di solito con lunghezza 1, cioè contenente un unico oggetto con TUTTE le informazioni relative alla località ricercata e strutturato come indicato nella pagina seguente.

In realtà **`geocoder.geocode`** restituisce una **classica promise** che, in alternativa alla callback, può essere gestita con il **`then`** e **`catch`** oppure con l'**`await`**. Notare però che al **`then`** non viene iniettato direttamente **`results`**, ma **`results`** viene inglobato all'interno di un ulteriore oggetto **response**:

```
promise.then(function (response){
    console.log(response.results)
})
```

Struttura del vettore results[ ] restituito da geocoder.geocode()


```
[  {
  "place_id":"ChIJa0Nw_KtZzRIRXqu7iJC19CA",
  "formatted_address"  :  "Via S. Michele, 68, 12045 Fossano CN, Italia",
  "geometry"  :  {"location" : {oggetto LatLng}  },

  "types"  :  ["street_address"] ,
  "address_components":    [
     {"long_name":"68",  "short_name":"68", "types":["street_number"]},
     {"long_name":"Via San Michele", "short_name":"Via S. Michele", "types":["route"]},
     {"long_name":"Fossano",  "short_name":"Fossano", "types":["locality","political"]},
     {"long_name":"Fossano", "short_name":"Fossano","types":["administrative_area_level_3","political"]},
     {"long_name":"Provincia di Cuneo", "short_name":"CN", "types":["administrative_area_level_2","political"]},
     {"long_name":"Piemonte", "short_name":"Piemonte", "types":["administrative_area_level_1","political"]},
     {"long_name":"Italia", "short_name":"IT", "types":["country","political"]},
     {"long_name":"12045", "short_name":"12045", "types":["postal_code"]}
   ],
  "location_type":"ROOFTOP",
  "viewport": {"south":44.5541030197085,  "west":7.735194019708501,
                "north":44.55680098029149, "east":7.737891980291465  }
}  ]
```


Il campo "**address_components**" contiene singolarmente tutte le varie componenti dell'indirizzo, sia in formato long_name (**Italia**) che in format short_name (**IT**)

Il campo "**geometry.location**" contiene le coordinate GPS espresse come oggetto **LatLng**.

Esempio

```
let address = "Fossano Via San Michele 68";
let geocoder = new google.maps.Geocoder();
geocoder.geocode( {'address': address}, function(results, status) {
     if (status == google.maps.GeocoderStatus.OK){

          disegnaMappa(results[0].geometry.location);
     }
     else
          alert("Stringa immessa non valida");
});
```

La funzione disegnaMappa() potrà istanziare un oggetto **Map** e disegnare una Google Map centrata nella posizione LatLng ricevuta come parametro.

Problema: Se si lanciano più 10 geocode() consecutivi le richieste successive vanno in errore. Per evitare questo problema occorre intervallare le richieste di almeno 500 ms, senza eseguire all'interno del ciclo altri metodi di accesso alle google maps:

```
setTimeout(function(){
    geocoder.geocode( {'address': address},  function(results, status) {
          // no other requests to google map here
        }
    )
}, 500 * i);
```

Calcolo delle routes fra due punti : l'oggetto DirectionsService

Il metodo **.route()** dell'oggetto **DirectionsService** riceve come parametro un json **routesOptions** contenente come parametri il punto di partenza ed il punto di arrivo e restituisce, tramite una funzione di callback, i migliori percorsi possibili tra il punto di partenza ed il punto di arrivo.

Punto di partenza e punto di arrivo possono essere espressi indifferentemente come:

- **Point**
- **LatLng**
- **stringhe alfanumeriche** che vengono automaticamente convertite in Point senza necessità di dover utilizzare l'oggetto Geocoder.

Utilizzo

```
let directionsService  = new google.maps.DirectionsService();
let routesOptions = {
   origin: partenza,
   destination: arrivo,
   travelMode: google.maps.TravelMode.DRIVING,  // default
   provideRouteAlternatives:true,               // default=false
   avoidTolls:false,                   // default (non evita pedaggi)
}
directionsService.route(routesOptions, function(directionsRoutes){
   let googleMap =  new google.maps.Map(div, mapOptions);
   if (directionsRoutes.status == google.maps.DirectionsStatus.OK)
      console.log(directionsRoutes.routes)
});
```

Alla funzione di callback del metodo **route()** viene iniettato un parametro **directionsRoutes** che è un json contenente, fra gli altri, i due seguenti campi principali:

status che può essere `google.maps.DirectionsStatus.OK / NOK`
routes **che è un vettore enumerativo di routes**, contenente TUTTE le routes individuate, in ordine di preferenza decrescente. Ogni route a sua volta può essere suddivisa in più tratte sequenziali dette **legs**

Anche il metodo **directionsService.route()** restituisce una **classica promise** che può essere gestita con il **then** e **catch** oppure con l'**await.**
Il valore iniettato al **then** è esattamente **directionsRoutes.**

Nota
Il metodo **.route()** impiega un certo tempo ad eseguire l'elaborazione, per cui se l'oggetto **Map** viene costruito prima di lanciare il metodo route(), la mappa viene visualizzata per un istante per poi essere sovrascritta dalla nuova mappa centrata sul percorso.
Per cui l'oggetto **Map**() deve essere istanziato solo SOLO all'interno della callback del metodo `route()`

Visualizzazione delle routes fra due punti: l'oggetto DirectionsRenderer

E' l'oggetto che consente di visualizzare graficamente le **routes** restituite dall'oggetto precedente.

Occorre istanziare un oggetto `DirectionsRenderer` per ogni route che si intende visualizzare.
Inoltre ogni oggetto **`DirectionsRenderer`** deve avere un proprio json di **`renderOptions`**.
Se si modificano le option anche dopo che il percorso è stato tracciato, le variazioni si ripercuotono graficamente sul percorso già tracciato.

In fase di istanza occorre passare a `DirectionsRenderer` le seguenti `renderOptions` (facoltative)

```
let renderOptions = {
   polylineOptions: {
      strokeColor : "#44F",   // colore del percorso
      strokeWeight : 6,       // spessore
      zIndex : 100            // posizionamento
   }
}

let directionsRenderer = new google.maps.DirectionsRenderer(renderOptions)
directionsRenderer.setMap(googleMap)    // Collego il renderer alla mappa
directionsRenderer.setRouteIndex(0)
directionsRenderer.setDirections(directionsRoutes)
```

Il metodo **`.setDirections()`** è quello che fisicamente disegna il percorso. Riceve come parametro l'intero oggetto **`directionsRoute`** restituito dal metodo precedente **`directionsService.route()`** e visualizza sulla mappa la **`route`** indicata dall'opzione **routeIndex** assegnata nella riga antecedente. Questa opzione per default vale 0 per cui, per default, viene visualizzata la sola route principale.

Pannello di informazioni

```
let panel = document.getElementById("panel")
panel.innerHTML=""
directionsRenderer.setPanel(panel)  // pannello con le indicazioni stradali
```

Distanza fra due punti e Tempo di percorrenza

Per calcolare la distanza fra due punti esistono due possibili alternative.

1) Distanza stradale sulla base del percorso calcolato dal metodo **`.route()`** di **`directionsService`**, che restituisce le varie possibile **`routes`** tra due punti.
Si prende in considerazione la route desiderata, cioè **`routes[i]`** e, al suo interno, la prima tratta (legs[0]), supponendo che la route sia costituita da una unica tratta come quasi sempre avviene.

```
let directionsRenderer = new google.maps.DirectionsRenderer(renderOptions)
directionsRenderer.setMap(googleMap)
directionsRenderer.setRouteIndex(0)
directionsRenderer.setDirections(directionsRoutes);

alert(directionsRoutes.routes[i].legs[0].distance.text);
alert(directionsRoutes.routes[i].legs[0].duration.text)
```

2) Distanza in linea d'aria (`great-circle distance`). calcolabile tramite il seguente metodo statico:

```
distance = google.maps.geometry.spherical.computeDistanceBetween (partenza, arrivo)
```

Occorre però aggiungere la seguente libreria nel link alle google.maps:
`https://maps.googleapis.com/maps/api/js?`**`libraries=geometry`**`&key=.........`
I parametri devono essere dei Point. Non sono accettate stringhe alfanumeriche

Lettura della posizione corrente

HTML5 mette a disposizione un nuovo oggetto **navigator.geolocation** che, tramite i metodi **`.getCurrentPosition()`** e **`.watchPosition()`** consente di ottenere le coordinate della posizione corrente. Nelle specifiche del W3C non viene fatta nessuna menzione in merito ai meccanismi che il browser deve utilizzare per recuperare questi dati, per cui ogni piattaforma può utilizzare al meglio le informazioni provenienti dal device. Normalmente il browser procede in questo modo:

- Se è presente un sensore GPS, la lettura viene eseguita direttamente dal sensore
- Viceversa il posizionamento viene seguito sulla base dell'ip della connessione internet, con eventuale ricerca su database pubblici gestiti dai vari provider che memorizzano l'area di assegnazione degli ip. Chrome utilizza anche altri criteri legati all'account google.

Per proteggere la privacy dell'utente è inoltre fatto divieto di utilizzo delle suddette informazioni senza preventivo consenso esplicito..

- Nel caso dei browser la richiesta di consenso viene visualizzata in automatico in corrispondenza dell'utilizzo dell'oggetto **navigator.geolocation.** Nel caso in cui l'utente neghi il consenso viene automaticamente richiamata la procedura di errore con codice di errore 1.
- Nel caso delle applicazioni android, è l'applicazione che chiede il consenso all'atto dell'installazione Se l'utente nega il consenso quella parte di applicazione non potrà essere eseguita.

I metodi dell'oggetto navigator.geolocation

`.getCurrentPosition(onSuccess, onError, options)` restituisce la posizione corrente. In caso di successo viene richiamata la funzione indicata come primo parametro, in caso di errore viene richiamata la funzione indicata come secondo parametro.

`.watchPosition(onSuccess, onError, options)` Lancia la lettura su un thread separato e legge con continuità fino a quando il thread non viene terminato. La frequenza delle letture dipende dal sensore e non è configurabile dall'utente. Se si desidera una lettura a cadenze regolari l'alternativa migliore è quella di utilizzare getCurrentPosition() all'interno di un setTimeout(). watchPosition restituisce un **`watchID`** identificativo del thread di lettura .

`.clearWatch(watchId)` consente di arrestare il thread associato al `watchId` indicato

Il terzo parametro **`options`** contiene i seguenti tre campi:

```
let options = {
    enableHighAccuracy: true/false
    timeout: 7000,
    maximumAge: 0,
    frequency : 10000          // deprecata (ignorata)
```

`enableHighAccuracy` I sensori possono restituire le posizioni GPS in modo più approssimativo (7 cifre, Based GPS feature) oppure in modo più accurato (Built-in GPS feature) però con una maggiore lentezza ed un maggior consumo di risorse (energia).
Questa proprietà consente di impostare quale modalità utilizzare. Il default è false e va benissimo nella maggior parte dei casi, a meno che non servano altissimi livelli precisione.

C'è però ancora un piccolo problema. Alcuni SO dispongono soltanto di una delle due modalità per cui se la richiesta usa una opzione diversa vanno in errore. La documentazione consiglia di utilizzare false e, in caso di errore, riprovare con true.

`timeout` (ms) indica il tempo max di attesa della risposta. Se la risposta non arriva entro questo tempo viene richiamata la procedura onError. Il valore di default è infinito, nel qual caso il metodo getCurrentPosition non ritorna nulla.

`maximumAge` (ms) indica al sensore un tempo max di cache dei valori restituiti nell'ultima lettura effettiva. Il default è 0 e significa che si chiede al sensore di restituire sempre valori effettivi.

frequency nel caso di watchPosition doveva indicare il tempo tra una lettura e l'altra in assenza di variazione delle posizione. E' però stata dichiarata obsoleta nelle API v3 e non più supportata.

function onSuccess(position){ }

riceve come parametro un oggetto **position** contenente i seguenti parametri.

coords.latitude	double -> decimal degrees
coords.longitude	double -> decimal degrees
coords.altitude	double or null -> meters (solo per sensori. Nel caso di indirizzo IP è null)
coords.accuracy	double -> meters (es: ± 100.000 metri nel caso dell'indirizzo IP)
coords.altitudeAccuracy	meters -> (es: ± 100 metri)
coords.speed	double or null -> meters/second velocità con cui il dispositivo si sta muovendo
timestamp	long -> timestamp unix espresso in msec a partire dal 1/1/1970

function onError(err){ }

riceve come parametro un oggetto **err** contenente i seguenti parametri:

err.code codice dell'errore
err.message messaggio dell'errore

Possibili codici di errore :

- PERMISSION_DENIED (1) if the browser denies you access to their location.
- POSITION_UNAVAILABLE (2) if the network is down or the positioning satellites can't be contacted.
- TIMEOUT (3) if the network is up but it takes too long to calculate the user's position.

Esempio completo

```
function onSuccess(position) {
    Per poter passare l'oggetto position a Map occorre convertirlo in LatLng
    let currentPos = new google.maps.LatLng(position.coords.latitude,
                                            position.coords.longitude);
    // oppure
    // mapOptions.center.lat = position.coords.latitude;
    // mapOptions.center.lng = position.coords.longitude;

    if(!mapID){  // il primo giro
        let wrapper = document.getElementById("wrapper");
        let mapOptions = {
            center:currentPos,
            zoom: 16 }
        mapID = new google.maps.Map(wrapper, mapOptions);

        marker = new google.maps.Marker({
           position: currentPos,
           title: "Questa è la tua posizione!",
           map: mapID
        });
    }
    else{
        marker.setPosition(currentPos)  // sposto il marker
        map.setCenter(currentPos)
    }
}

function onError(err) { alert("ERRORE:" + err.code + " - " + err.message) }
```